FORMOSA CRYPTO

Completing the Chain:

Verified Implementations of Hash-Based Signatures and Their Security

Manuel Barbosa François Dupressoir **Rui Fernandes**
Andreas Hülsing Matthias Meijers Pierre-Yves Strub

Repository

Paper

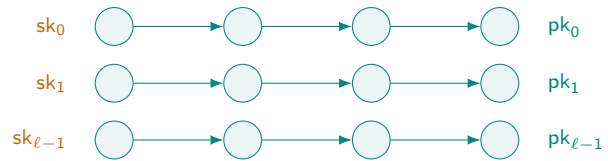
CHES 2026

XMSS: Overview

- XMSS [BDH11] is a stateful hash-based signature scheme.
- Security relies on the security of the underlying hash function.
- XMSS combines a one-time signature scheme (WOTS^+) with a Merkle tree.
  - The root of the Merkle tree is the public key of the XMSS instance.
  - Each leaf corresponds to a (compressed) WOTS^+ public key.
  - A Merkle tree of height h has 2^h leaves, and thus allows 2^h signing 2^h messages.

WOTS^+: Key Generation

1. Generate ℓ secret n-byte values $\mathsf{sk}_0, \ldots, \mathsf{sk}_{\ell-1}$ (randomly or pseudorandomly).
2. Each value starts a hash chain: apply $w-1$ steps to obtain $\mathsf{pk}_j = \mathsf{chain}(\mathsf{sk}_j, 0, w-1)$.
3. The WOTS^+ public key is the array of all ℓ chain endpoints.



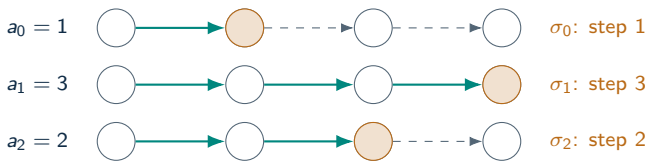
Example: $w = 4$, three steps per chain (selected chains shown).

n: element size in bytes; w: Winternitz parameter; ℓ: number of message and checksum digits.

Each step uses a hash key and bitmask derived from public SEED and the keypair, chain, and step address.

WOTS$^+$: Signature Generation

1. Encode the n-byte message digest D as base-w digits $m_0, \ldots, m_{\ell_1-1}$.
2. Append the base-w digits of the checksum $C = \sum_{j=0}^{\ell_1-1}(w-1-m_j)$, giving ℓ digits $a_j \in \{0, \ldots, w-1\}$.
3. Reveal one node per chain: $\sigma_j = \text{chain}(\text{sk}_j, 0, a_j)$. The signature is $(\sigma_0, \ldots, \sigma_{\ell-1})$.



Example: $w = 4$; gold nodes form the signature (selected chains shown).

Verification completes each chain with $w - 1 - a_j$ steps to recover its endpoint.

Each WOTS$^+$ keypair signs at most one message digest.

XMSS: Key Generation

1. Start from 2^h WOTS^+ keypairs: one for each leaf of the tree.
2. Compress each WOTS^+ public key via an L-tree (LTree) to an n-byte value. These values are the leaves.
3. Hash pairs of nodes upward, level by level, to obtain the n-byte root.

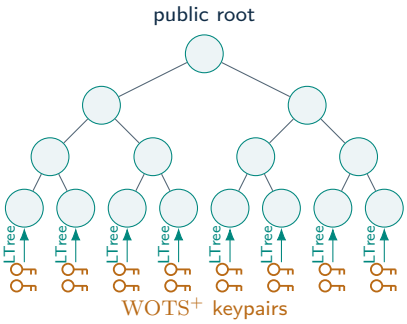
Scheme parameters

h: tree height (2^h leaves).
n: hash output length in bytes.

Public key: $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, with public seed SEED.

Secret key: $\mathsf{SK} = (i, \{\mathsf{sk}_j^{\mathrm{WOTS}^+}\}_{j=0}^{2^h-1}, \mathsf{SK}_{\mathrm{PRF}}, \mathsf{PK})$.

$\mathsf{SK}_{\mathrm{PRF}}$: secret key for message randomization.

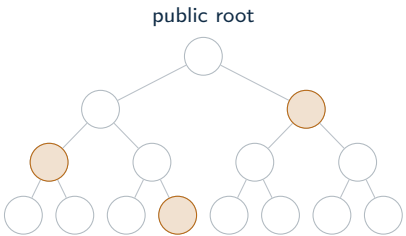


Example: $h = 3$

XMSS: Signature Verification

Inputs: message M, PK $= (\text{root}, \text{SEED})$, $\sigma = (i, R, \sigma_{\text{WOTS}^+}, \text{auth}_i)$, with $0 \leq i < 2^h$.

1. Recompute the message digest $D = H_{\text{msg}}(R \parallel \text{root} \parallel i, M)$.



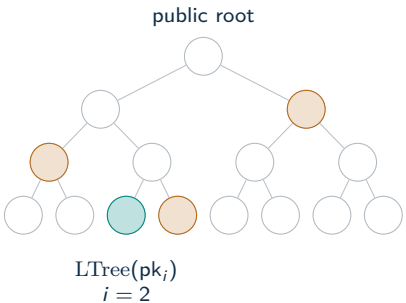
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS^+}}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the $\mathrm{WOTS^+}$ chains from $\sigma_{\mathrm{WOTS^+}}$ to recover pk_i.



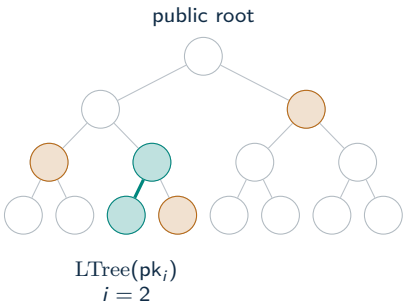
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, PK $= (\text{root}, \text{SEED})$, $\sigma = (i, R, \sigma_{\text{WOTS}^+}, \text{auth}_i)$, with $0 \leq i < 2^h$.

1. Recompute the message digest $D = H_{\text{msg}}(R \parallel \text{root} \parallel i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.
3. LTree compression gives leaf i.



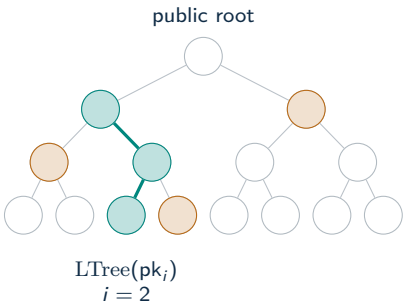
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS}^+}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the WOTS^+ chains from σ_{WOTS^+} to recover pk_i.
3. LTree compression gives leaf i.
4. Hash upward with the h authentication siblings. Bits of i determine left/right order.



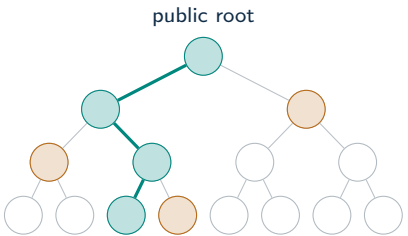
Gold: supplied authentication siblings.

Teal: reconstructed path.

XMSS: **Signature Verification**

Inputs: message M, $\mathsf{PK} = (\mathsf{root}, \mathsf{SEED})$, $\sigma = (i, R, \sigma_{\mathrm{WOTS^+}}, \mathsf{auth}_i)$, with $0 \le i < 2^h$.

1. Recompute the message digest $D = H_{\mathrm{msg}}(R \,\|\, \mathsf{root} \,\|\, i, M)$.
2. Complete the $\mathrm{WOTS^+}$ chains from $\sigma_{\mathrm{WOTS^+}}$ to recover pk_i.
3. LTree compression gives leaf i.
4. Hash upward with the h authentication siblings. Bits of i determine left/right order.
5. Accept if the reconstructed root equals root in the public key.

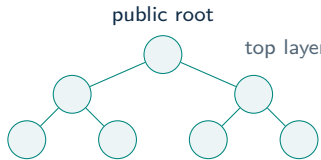


Gold: supplied authentication siblings.

Teal: reconstructed path.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- Split total height h into d layers of XMSS trees, each of height h/d.

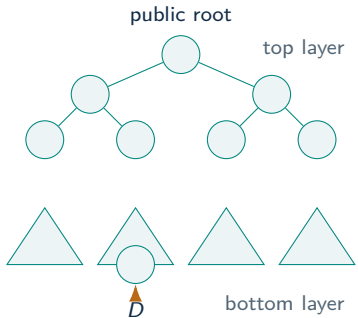


Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- Split total height h into d layers of XMSS trees, each of height h/d.
- Bottom-layer WOTS^{+} keys sign message digests.

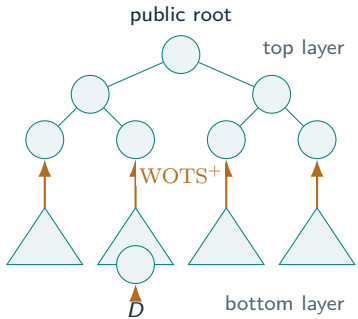


Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

- Split total height h into d layers of XMSS trees, each of height h/d.
- Bottom-layer WOTS^{+} keys sign message digests.
- Upper-layer WOTS^{+} keys sign the roots of trees in the layer below.



Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

$\mathrm{XMSS}^{\mathrm{MT}}$: Multi-Tree Variant

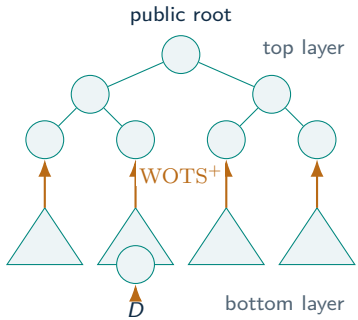
- Split total height h into d layers of XMSS trees, each of height h/d.
- Bottom-layer WOTS^{+} keys sign message digests.
- Upper-layer WOTS^{+} keys sign the roots of trees in the layer below.
- Verify layer by layer up to the single public root.

Tradeoff

2^h messages; smaller trees to generate.

Signature: d WOTS^{+} signatures and h path nodes.

For $d = 1$, $\mathrm{XMSS}^{\mathrm{MT}}$ is functionally equivalent to XMSS.



Example: $h = 4$, $d = 2$.

Gold arrows: WOTS^{+} signs a digest or root.

Jasmin [ABB+17, ABB+20]

- High-level assembly language designed for cryptography implementations
- High-level zero-cost abstractions: variables, arrays
- Control over low-level details (e.g. instruction selection, loop unrolling)
- Suported architectures: x86_64, ARMv7M, ARMv8-A, RISC-V

Jasmin [ABB+17, ABB+20]

- Stack zeroization [OBG+24]
- (Static) Safety checker
- Compiler correctness proved in Rocq
- Extraction to EasyCrypt
- Speculative constant-time:
  - Spectre v1 [SBG+23] and other spectre variants [ABC+25]
  - Compilation preserves SCT [ABB+25].

Compiler Correctness & Constant-Time Preservation

Compilation preserves the semantics of *safe* source programs and constant-time behavior: functional correctness and constant-time guarantees established at source level carry to the assembly level.

EasyCrypt

- Interactive proof assistant.

Verification Workflow

EasyCrypt

Jasmin

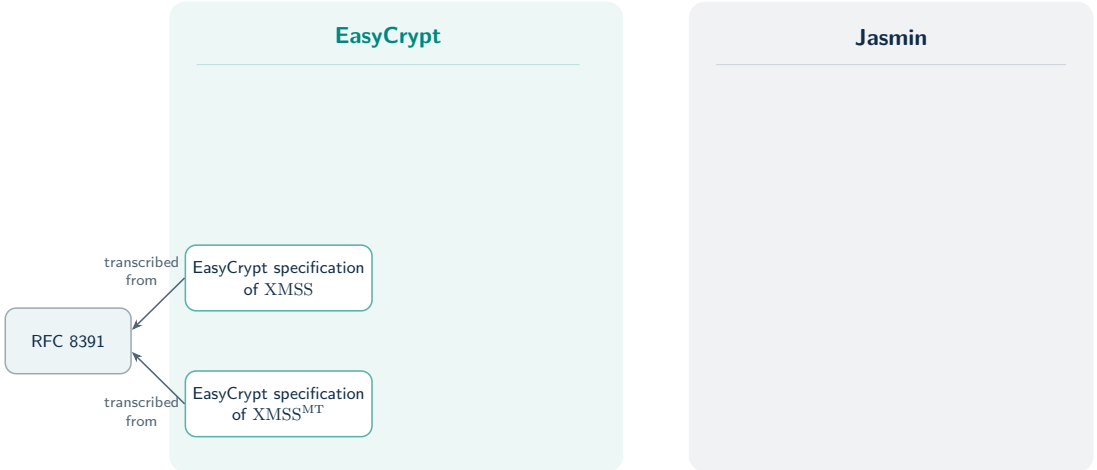
Verification Workflow

RFC 8391

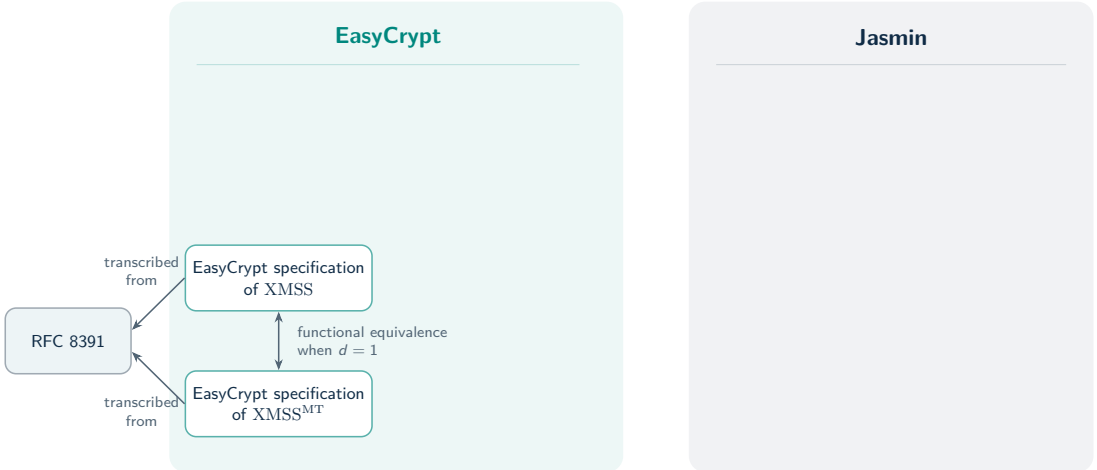
EasyCrypt

Jasmin

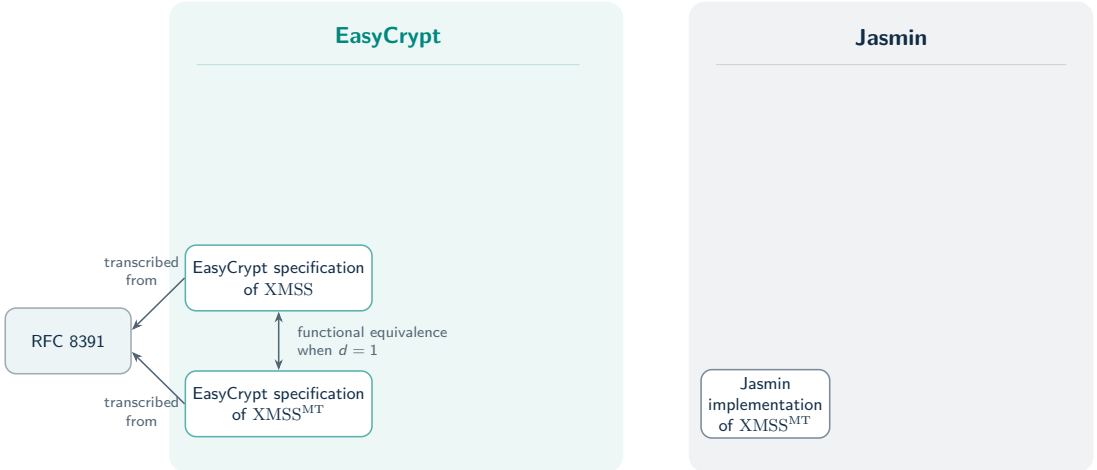
Verification Workflow



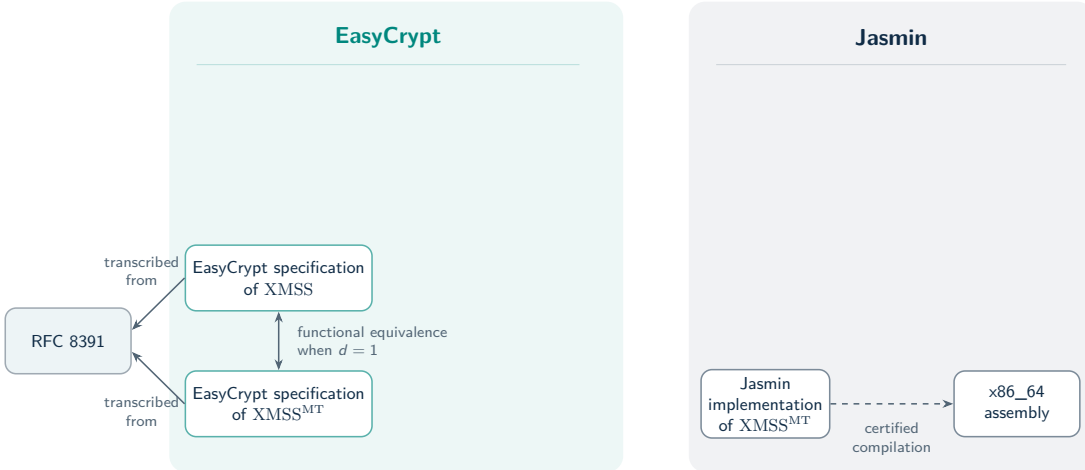
Verification Workflow



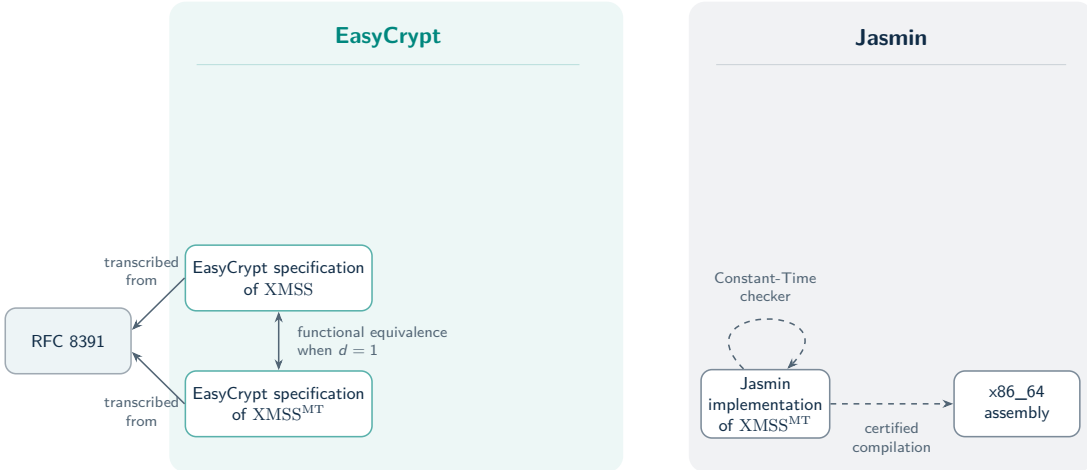
Verification Workflow



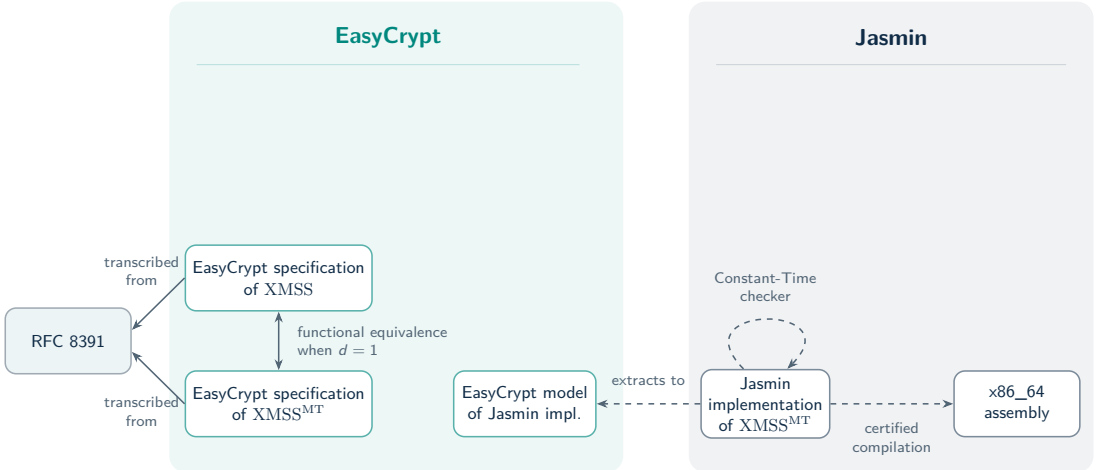
Verification Workflow



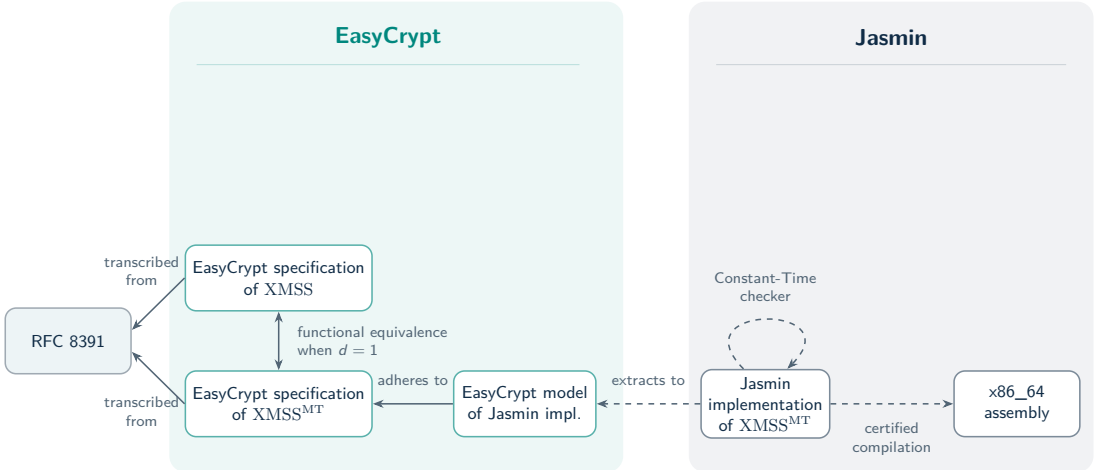
Verification Workflow



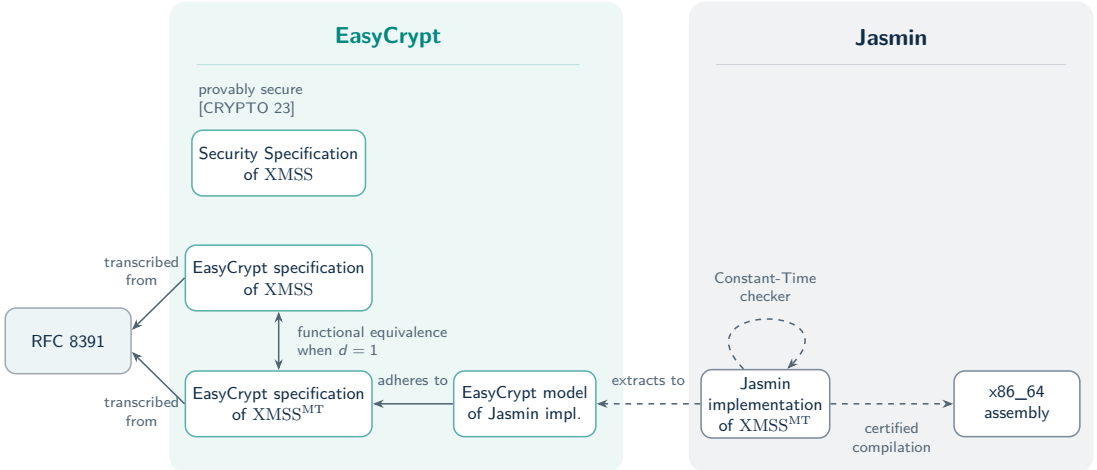
Verification Workflow



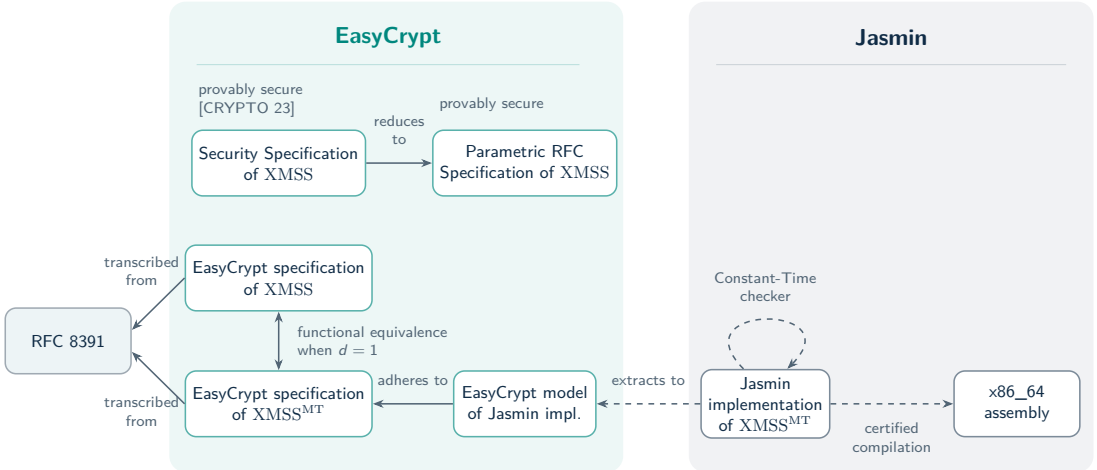
Verification Workflow



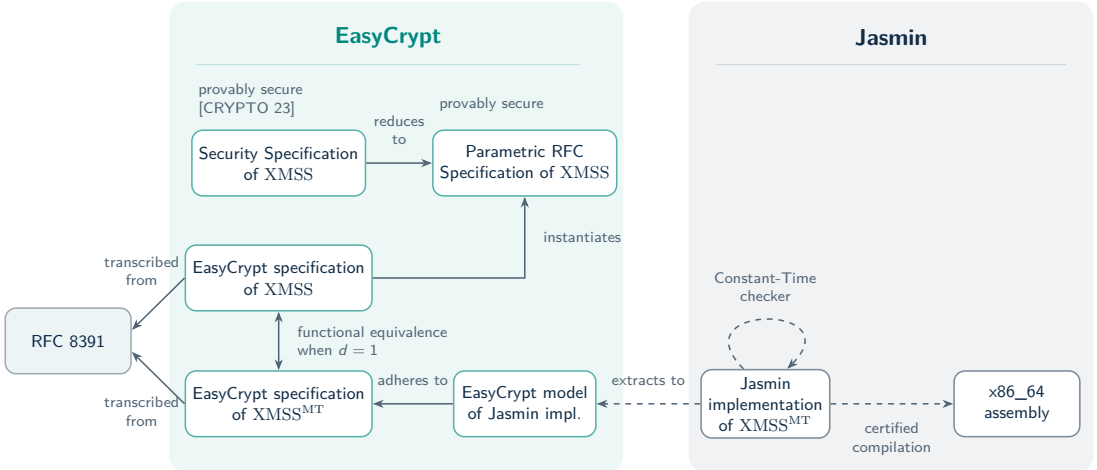
Verification Workflow



Verification Workflow



Verification Workflow



Trusted Computing Base

- Correctness of the SHA-256 implementation is outside the scope of the proof

```
op Hash : W8.t list -> nbytes. (* Abstract operator *)

axiom hash_128 (x : W8.t Array128.t):
  phoare[
    M(Syscall).hash_plen_3n____sha256:
    arg.`2 = x
    ==>
    to_list res = NBytes.val (Hash (to_list x))
  ] = 1%r.
```

Figure: EasyCrypt axiom stating the correctness of SHA-256.

Trusted Computing Base

- Jasmin compiler: unverified OCaml code (parsing and typing)
- Jasmin extraction to EasyCrypt
- EasyCrypt codebase
- SMT Solvers (Z3, CVC4 and Alt-Ergo)

Implementation

- Array sizes are fixed at compile time (one function per size)

Implementation: Performance

	C Ref	Jasmin (abs, %)	loop zeroization (abs, %)	unrolled zeroization (abs, %)
Keygen	10092.74	8263.54 (-18.12%)	8293.61 (+0.36%)	8312.35 (+0.59%)
Sign	20251.01	24775.17 (+22.34%)	24872.05 (+0.39%)	24927.53 (+0.61%)
Verify	10.35	8.47 (-18.14%)	8.48 (+0.13%)	8.95 (+5.69%)

Table: Performance Comparison between C and Jasmin implementations in 10^6 clock cycles (column #2). Zeroization overhead w.r.t. Jasmin baseline (columns #3 and #4).

QUESTIONS?

References I

[ABB+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub.
Jasmin: High-assurance and high-speed cryptography.
In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017.

[ABB+20] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub.
The last mile: High-assurance and high-speed cryptographic implementations.
In *2020 IEEE Symposium on Security and Privacy*, pages 965–982. IEEE Computer Society Press, May 2020.

[ABB+25] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, and Vincent Laporte.
Preservation of speculative constant-time by compilation.
Proc. ACM Program. Lang., 9(POPL):1293–1325, 2025.

References II

[ABC+25] Santiago Arranz-Olmos, Gilles Barthe, Chitchanok Chuengsatiansup, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Peter Schwabe, Yuval Yarom, and Zhiyuan Zhang.

Protecting cryptographic code against spectre-rsb: (and, in fact, all known spectre variants).

In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 933–948. ACM, 2025.

[BDH11] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing.

XMSS - A practical forward secure signature scheme based on minimal security assumptions.

In Bo-Yin Yang, editor, *Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings*, volume 7071 of *Lecture Notes in Computer Science*, pages 117–129. Springer, 2011.

References III

[OBG+24] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe.

High-assurance zeroization.

IACR TCHES, 2024(1):375–397, 2024.

[SBG+23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean.

Typing high-speed cryptography against spectre v1.

In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023.